← 返回记忆卡 打印 / 另存为 PDF ☑ 连「理解铺垫」一起印

点「打印 / 另存为 PDF」，在弹出窗口里把「目标」选为「另存为 PDF」即可保存。 每张卡的**结论**单独成块——想自测就把结论那半栏用纸盖住，看着标题先自己背一遍。 默认不印「理解铺垫」（那是理解用的长文，会让页数翻几倍）；需要就勾上面的选项。

计算机组成原理 · 记忆卡

codebrick.tech · 用户ID 14127 · 2026-10-27 · 共 20 张

计算机系统概述 1 张

计算机系统层次结构 中频 定义锚点

冯·诺依曼机核心 = 「存储程序」工作方式：

- **存储程序**：程序和数据预先存入主存，启动后 CPU 自动逐条取指执行，无须人工干预。
- **五大部件**：运算器、控制器、存储器、输入设备、输出设备。现代机把运算器+控制器集成为 CPU。
- **指令与数据同等地位**：都用二进制存于同一主存、按地址访问；位串本身无标记，靠 CPU 上下文（PC/状态）区分取的是指令还是数据。
- **演进**：早期「以运算器为中心」→ 现代「以存储器为中心」。
- 指令 = 操作码（做什么）+ 地址码（操作数在哪）。

机器级语言层次（只记分层，不展开）：

- 高级语言 → 汇编语言 → 机器指令 → 微指令，自上而下越来越接近硬件；硬件能直接执行的只有机器语言。

性能指标的定义辨析（只记「衡量什么、哪里失真」，公式代入归题）：

指标	衡量什么	失真点
CPI	执行一条指令平均所需的时钟周期数	随指令组合而变，换个目标代码序列就不同
MIPS	每秒执行多少百万条指令（定点）	不能跨指令集比较；厂商常把峰值 MIPS 当实测值
MFLOPS	每秒多少百万次浮点运算	按操作次数而非指令条数计，只反映浮点负载

- MIPS 不可比的根因：机器指令集不同、单条指令的功能不同，M1 上一条指令干的活在 M2 上可能要多条指令才能完成，指令条数本身就不是同一把尺。
- 时钟周期、指令条数、CPI 三者相互制约：指令条数最少的程序不一定执行得最快，主频高也不等于速度快，三个因素必须同时看。
- Amdahl 定律只记定性结论：整体加速比受限于不可加速部分，存在上界——改进部分占比 t 时，把它加速到无穷倍，整体加速比也只趋于 1/(1-t)。

这章是计组最薄的一章，记忆点就是冯·诺依曼机的几条「定义性结论」，背的是「是什么」而不是任何计算。

最该钉死的两个易混点：

- 「指令和数据形式上无区别」——很多人以为 CPU 区分得了，其实位串本身没有任何标记，靠的是程序计数器和当前执行阶段（取指 vs 取数）来区分。
- 「数据都在指令中给出」是**错的**——只有立即寻址的数据在指令里，直接/间接/寄存器/变址等寻址方式下数据都另存在内存或寄存器。这是冯·诺依曼机叙述题的经典陷阱（co-2019-12 的正确答案就考这个）。

怎么记五大部件：运算器+控制器（合成 CPU）+ 存储器 + 输入 + 输出。注意「以运算器为中心」是早期结构，现代机演进为「以存储器为中心」，总线把各部件挂在一起。

软硬件分层只记一个方向感：高级语言要先被翻译（编译器/解释器）成机器语言硬件才认。具体编译四阶段（预处理→编译→汇编→链接）属于流程辨析，记顺序即可（co-2022-20 考过），求值/工具链细节看下方博客精讲。

性能指标这块，考的是「定义」不是「算式」：真正拉分的是判断题式的一句话——「MIPS 值高的机器一定更快吗」。答案是不一定，而且理由要说到点子上：MIPS 数的是**指令条数**，可不同机器的指令集不同、同名指令干的活也不同，M1 上一条指令完成的功能 M2 上可能要拆成好几条，指令条数在两台机器之间根本不是同一把尺；再加上两机的 CPI 和时钟周期也不同，同一条指令耗时也不一样。所以 **MIPS 只在同一指令集内部横向比较才有意义**，跨指令集比就是无效比较。

同理，主频高不等于快、指令条数少也不等于快。执行时间由指令条数、CPI、时钟周期三者共同决定，而这三个量彼此制约（比如为提主频而加深流水线，往往把 CPI 拱上去），只盯其中一个必然被坑。CPI 也不是机器的固有常数——它随程序的指令组合变化，同一台机器跑不同目标代码序列，综合 CPI 就不同。MFLOPS 与 MIPS 的差别只有一处要记：它数的是**浮点操作次数**而不是指令条数，因此只对浮点密集的负载有代表性。

⚠️ **两个 MIPS 别混**：这里说的 MIPS 是性能指标（million instructions per second，每秒百万条指令）； `cisc-risc` 卡里作为 RISC 典型代表出现的 MIPS 是一种**指令集架构**的名字。缩写撞车而已，两者毫无关系，题干里出现 MIPS 先看它在讲速度还是在讲机器。

Amdahl 定律只背一个直觉：系统整体性能被没被加速的那部分拖住。哪怕把占总时间 90% 的部件提速 10 倍，整体也只快 5 倍出头；再把改进倍数推到无穷大，整体加速比也只能逼近 1/(1−t) 这个天花板。推论很实用——优化只占极小比例的部分（比如异常处理程序），对整机性能提升几乎为零。具体倍数代入是计算题，卡里只锚「存在上界、上界由不可加速部分决定」。

数据表示与运算 3 张

定点数编码（原/反/补/移） 高频 对比表

原 / 反 / 补 / 移四码——**定义与关系结论**（具体求值靠题）：

编码	正数	负数	0 的表示	表示范围（n 位含符号）
原码	符号位 0 + 绝对值	符号位 1 + 绝对值	+0/-0 **两个**	$-(2^{n-1}-1) \sim +(2^{n-1}-1)$
反码	同原码	符号位外**按位取反**	+0/-0 **两个**	$-(2^{n-1}-1) \sim +(2^{n-1}-1)$
补码	同原码	**反码 + 1**	**唯一**	$-2^{n-1} \sim +(2^{n-1}-1)$
移码	补码符号位取反	补码符号位取反	**唯一**	$-2^{n-1} \sim +(2^{n-1}-1)$

关系链（死记）：负数 反码 = 原码符号位外取反 → 补码 = 反码 + 1 → 移码 = 补码**符号位取反**。

- **正数原/反/补三码相同**，移码与补码只差符号位。
- 补码比原码/反码**多表示一个最小负数** -2^{n-1}（其位模式 $10\ldots0$ 无对应原码/反码）。
- 移码用于浮点**阶码**，真值越大移码（按无符号读）越大，**便于直接比较大小**。

补码加减的溢出判断三法（同一件事的三个观察角度，具体演算归题）：

- **双符号位**（变形补码，模 4）：两位符号位不同即溢出——01 上溢、10 下溢；相同则无溢出。
- **进位异或**：符号位产生的进位 C_n 与最高数值位向符号位的进位 C_{n-1} 不同则溢出，即 $\mathrm{OF} = C_{n-1} \oplus C_n$。
- **符号比较**：两个加数同号、而与结果的符号位不同，则溢出。推论：异号两数相加永不溢出。

标志位 CF / ZF / SF / OF：

标志	含义	对谁有意义
CF	进位/借位（加法进位、减法不够减借位）	只对无符号整数
ZF	结果为全 0	无符号 / 带符号都有意义
SF	结果的最高位，即符号	只对带符号整数
OF	带符号整数运算溢出	只对带符号整数

四码的考点是「转换关系」和「适用场景」这些定性结论，**具体某个数求补码/求真值是计算，交给题和可视化**（这张卡只锚结论）。

怎么记不混：

- 一条链走到底：原 →（负数符号位外取反）→ 反 →（+1）→ 补 →（符号位取反）→ 移。记住「反取反、补加一、移翻符」。
- **正数原/反/补三码相同**，三者的差异全在负数；移码不论正负都和补码差符号位。
- 「补码多一个数」是命题高频点：n 位补码范围 $[-2^{n-1}, 2^{n-1}-1]$，比原码/反码多出最小负数 -2^{n-1}，因为补码 0 只占一个编码、省出来的位模式 $10\ldots0$ 给了它。
- 移码为什么用于阶码：它把真值大小顺序和无符号位模式顺序对齐，浮点比较大小时直接按位模式比即可，不用还原真值。

「3 个 1、5 个 0 的补码求最小整数」「由补码还原真值」这类是位模式求值（co-2015-13 即此型），属计算，看下方可视化逐步推。卡里只留四码的定义、关系与范围结论。

溢出判断三法怎么挑着用：手算判断用「双符号位」最快，两位符号位一眼扫过去不同就是溢出，左符是真符号、右符专门用来看溢出；看电路和数据通路的题用「进位异或」，硬件里的 OF 就是这么产生的；口算估形势用「符号比较」——异号两数相加，和的绝对值只会比两个加数都小，根本冲不出范围，所以只有同号相加才需要担心溢出。三条判据判的是同一件事，答案必然一致，对不上就是自己算错了。

四个标志位最爱考的一句：**OF 只对带符号整数有意义，CF 只对无符号整数有意义**。一条加法指令执行完，四个标志是一起产生的，硬件并不知道你打算把这串位模式当有符号还是无符号来读——是「怎么读」决定了该看哪个标志。所以「CF=1 就是溢出了」是错的，那只说明按无符号读时产生了进位；同理带符号运算时盯着 CF 看也没有意义。

> 具体某组数的补码加减演算、逐位推进位、判这一次到底溢没溢出，属计算，归题；卡里只留三条判据的形式和四个标志各自的含义与适用对象。

IEEE754 浮点表示 高频 定义锚点

IEEE 754 浮点格式——**结论锚点**（具体编码换算靠题）：

项	单精度 float	双精度 double
总位宽	**32 = 1 符号 + 8 阶码 + 23 尾数**	**64 = 1 + 11 + 52**
偏移量（bias）	**127**	**1023**

- 阶码用**移码（偏置码）**：实际指数 = E − bias；用移码便于直接按位模式比较大小。
- 尾数有**隐含位 1**：规格化数尾数实为 $1.M$，最高位的 1 恒为 1 故不存储，省出 1 位精度。
- **阶码全 0**：尾数=0 → ±0；尾数≠0 → 非规格化数。
- **阶码全 1**：尾数=0 → ±∞；尾数≠0 → NaN。

IEEE 754 这张卡只背「格式骨架 + 特殊值判定」这些结论；**给定十进制求二进制编码、对阶/规格化/舍入运算全是计算**，交给题和可视化，卡里不演算。

怎么记位宽：单精度「1-8-23」加起来 32，双精度「1-11-52」加起来 64。配套偏移量是 `127`（= $2^{8-1}-1$）和 `1023`（= $2^{11-1}-1$）——注意偏移是 **2^(k-1)−1**，比纯移码的 2^(k-1) 少 1，这是最爱挖的坑（误用 128 是 co-2025-13 的典型错因）。

隐含位 1 是必考送分点：规格化数的尾数前面有一个「默认存在、不存储」的 1，所以真实尾数是 `1.M` 而不是 `0.M`。还原真值漏掉它，系数就从 1.x 错成 0.x（co-2025-13 的一个错项正是漏隐含 1）。

特殊值用「阶码全 0 / 全 1」两道闸记：

- 阶码**全 0**：尾数也为 0 是 ±0；尾数非 0 是非规格化数（隐含位变 0、填补 0 附近的空隙）。
- 阶码**全 1**：尾数为 0 是 ±∞；尾数非 0 是 NaN。
- 一句话：「全 0 管最小那头（0/非规格化），全 1 管最大那头（∞/NaN）」。

> 十六进制↔真值的拆段、规格化数指数减偏移、补隐含 1 拼真值这些步骤，看下方可视化逐步走；卡里只锚位宽、偏移、隐含位、阶码全 0/全 1 这四组结论。

数据对齐与大小端 中频 对比表

① 大端 vs 小端（判定只看"低地址放的是哪一端"）

	大端 big endian	小端 little endian
低地址存放	最高有效字节 MSB	最低有效字节 LSB
内存转储拼真值	顺着拼	倒着拼
代表机器	MIPS、68k、SPARC；网络字节序	Intel 80x86、DEC VAX

- **变量的地址 = 它所占连续单元中最小的那个地址**，两种方式都一样；字节序只决定这个地址上放的是数据的哪一端。
- 判定代码： `int x = 1; *(char *)&x == 1` 成立即小端。
- 两种方式本身无优劣，**只在跨系统通信时必须转换**；机器码中的立即数、位移量字段同样按字节序摆放。

② 边界对齐规则

- 通用策略：**k 字节的数据存放在 k 的整数倍 地址上**（char 1、short 2、int/float 4、double 8）。
- ⚠️ 具体对齐到几字节**由 ABI 规范/编译器规定，不是物理定律**——题面给定的要求优先。
- 数组按 元素类型 的对齐要求（不是数组总长）；嵌套结构体按其内部最严格的成员。
- 对齐到 k 等价于**地址二进制低 $\log_2 k$ 位恒为 0**，这几位可以不存：指令按字（4B）对齐时，32 位地址空间的 PC 只需 30 位。

③ 结构体三条规则（内部插空 + 尾部插空）

	规则	后果
①	整个结构体的对齐要求 = 对齐最严格的成员	决定首地址与总大小的倍数
②	每个成员在满足自身对齐的前提下取最小可用偏移	内部插空
③	总大小必须是 自身对齐要求的倍数	尾部插空

- **尾部填充是为结构体数组存在的**：数组步长 = 含尾部填充的 sizeof，不是各成员大小之和。
- 成员按对齐要求 从大到小 排列，填充最少（只改声明顺序就能省空间）。

④ 跨边界访问的代价

- 存储器一次读写的是一整块（8 字节宽即 8i~8i+7）；数据跨了块边界就要 两次访存 再拼接。
- **MIPS、ARM 要求数据按自然边界对齐**（RISC-V 不作要求）；x86/IA-32 不对齐也能正确工作，**只是慢**。
- 字节序与对齐**互不相干**：字节序管数据内部的字节先后（体系结构定），对齐管数据整体落在哪个地址（ABI/编译器定）。

这张卡两件事：字节序怎么判不反，对齐怎么排不错。两者常在同一道题里同时出现，但它们是**两套独立规则**——先按对齐定出每个成员在哪个地址，再按字节序定这些地址里各放哪一字节。

小节① 大端小端怎么记不反：

- 抓一句口诀就够：**小端 = 低地址存低字节**（小对小），大端反之。"数据的地址"在两种方式下是同一个数——都是最小的那个单元地址，变的只是这个单元里装的是 MSB 还是 LSB。
- 最常见的错法出在**读内存转储**：题面按地址从小到大列出字节，那是物理顺序不是书写顺序。小端下必须倒着拼才是真值，顺着读出来的十六进制数是错的。
- 大端也叫网络字节序，所以跨系统传数据要各自转换；有些处理器（如 Alpha、PowerPC）两种都支持，加电启动时选定一种。

小节②③ 对齐到底在省什么：

- 省的是**访存次数**，不是空间——对齐反而要多花空洞。存储器是按块读写的，一个数据只要落在块内就一次取回，跨了边界就得取两次再拼。所以对齐的收益随存储器宽度走，跟"美观"无关。
- 三条规则的分工：① 决定整个结构体放在哪、总大小是谁的倍数；② 造成**内部插空**；③ 造成**尾部插空**。①② 保证单个变量内每个成员都对齐，③ 保证**结构体数组里第二个及以后的元素**也对齐——数组步长就是含尾部填充的 sizeof，不补尾部则后续元素整体错位、内部成员的对齐全部失效。
- 两个高频误判：**数组成员的对齐要求由元素类型决定**（ `char name[10]` 是 1 不是 10），**结构体取的是最大"对齐要求"而不是最大"成员大小"**（差别只出现在数组成员和嵌套结构体上）。
- 还有一条推论值得单独记：对齐等价于地址低几位恒为 0，硬件因此可以不存这几位。真题里"指令按字边界对齐 → PC 位数比字节地址少 2 位"考的就是它；顺带注意 IR 装的是整条指令，位数不会跟着缩。

⚠️ 减法：具体某个结构体的成员偏移量、sizeof、"某字节落在哪个地址"这类**代入演算一律归题**（已知各成员大小之和也算不出 sizeof，只能逐成员排偏移，没有捷径）。卡里只留字节序判定、对齐规则与三条结构体规则这层骨架。

> 上方真题正是这两条线：co-2012-15 / co-2020-14 / co-2025-15 考对齐 + 小端联合定位，co-2016-14 / co-2018-15 / co-2019-15 只考字节序方向，co-2016-18 考"字对齐 → PC 位数"，co-2017-13 考跨边界访问的存储周期数，co-2011-18 考"按边界对齐有利于流水线"。

存储器层次结构 5 张

Cache 映射方式 高频 对比表

Cache 三种映射方式对比（背特点，不背地址计算）：

方式	主存块能放到哪	冲突率	命中率	硬件
直接映射	只能进**固定一行**（块号 mod 行数）	高	低	最简单、命中判断快（只比 1 行）
全相联	任意块进**任意行**	**最低**	最高	最复杂、需全部比较器、查找慢、成本高
组相联	先映射到固定**组**、组内任意行	居中	居中	折中（k 路只需 k 个比较器）

- 关系：k 路组相联中 **k=1 退化为直接映射，组数 S=1（k=行数）退化为全相联**。
- 替换算法记名称与适用：**LRU**（命中率最高，有栈性质无 Belady 异常）/ **FIFO**（简单，可能有 Belady 异常）/ **随机**。直接映射无替换选择（位置唯一）。

三种映射的差别只在一句话：**一个主存块「允许进哪些 Cache 行」的自由度不同**。直接映射自由度最低（钉死一行），全相联最高（哪行都行），组相联折中（先定组、组内随便）。自由度越高，冲突越少、命中率越高，但比较器越多、硬件越贵越慢——这是一条贯穿始终的权衡轴。

怎么记不混：

- **直接映射**：块号 mod 行数，位置唯一 → 像「对号入座」，找得快但易撞座（不同块抢同一行相互驱逐，冲突高）。
- **全相联**：任意行都行 → 像「随便坐」，最不易撞但要把全场座位都查一遍（全部 tag 并行比较，比较器最多，常只用于 TLB 这种小 Cache）。
- **组相联**：先按组号 mod 组数定到某组，组内随便坐 → 「先定区域再随便坐」，现代 CPU 主流（2/4/8 路）。

替换算法只在「有多个候选行」时才需要，所以**直接映射根本不用替换算法**（位置唯一），全相联和组相联才用。LRU 替换最久没用过的、命中率最高；FIFO 替换最早进来的、实现简单但可能 Belady 异常。

> ⚠️ 地址位拆分（标记 / 行号或组号 / 块内地址各几位）、命中率与平均访问时间计算全部归题，卡里只对比「方式特点」。这是 co-storage 最高分章，但高分点都在那些换算上，靠刷题 + 可视化。

Cache 写策略 中频 对比表

写策略分两个**独立维度**：写命中怎么同步主存、写不命中要不要调入 Cache。

写命中：写回 vs 写直达

	写回（write-back）	写直达 / 直写 / 全写法（write-through）
主存同步时机	只写 Cache、置**脏位**，**替换时**才写回主存	写 Cache 时**同时**写主存
一致性	可能不一致	**始终一致**
主存访问量	少	多（常配**写缓冲器**缓解）
是否需脏位	**需要**（每行 1 位）	不需要

写不命中：写分配 vs 写不分配

写分配	写不分配
先把该块**调入 Cache** 再写	**只写主存**，不调入 Cache

常见组合：写回 + 写分配 / 写直达 + 写不分配。

写策略要回答两个互相独立的问题：① 写**命中**时，新数据怎么让主存「看见」？ ② 写**不命中**时，要不要先把这块调进 Cache? 两个维度各两种选择，理论上 2×2，但实际只有两套稳定组合。

写命中维度（写回 vs 写直达），顺着「访存次数 vs 一致性」这条轴记：

- **写回**：只动 Cache、给该行置「**脏位**」，等这行被替换、且脏位为 1 时才整体写回主存。多次写同一行只产生 1 次主存写 → 访存少、性能高；代价是 Cache 与主存**可能不一致**，且每行要多 1 位脏位。
- **写直达**（教材里也叫**直写**、**全写法**，见到别当成新概念）：写 Cache 的同时把这次写也发往主存 → Cache 与主存**始终一致**、不需脏位、硬件简单；代价是每次写都吃主存带宽，常配**写缓冲器**让 CPU 不必同步等待。

写不命中维度：**写分配**＝先调块进 Cache 再写（赌这块后面还会被反复访问）；**写不分配**＝直接写主存、不占 Cache 行（赌这是一次性写）。

为什么是这两套组合：写回靠「批量合并写回」赚性能，自然希望数据先进 Cache 多次合并 → 配**写分配**；写直达反正每次都要写主存，调进 Cache 没省到什么、还白占行 → 配**写不分配**。考试里出现「写回+写不分配」「写直达+写分配」这种奇葩搭配，基本都是干扰项。

> ⚠️ 「Cache 总容量 / 标记阵列位数」计算时，**写回每行多 1 位脏位、写直达没有**，是高频失分点——但具体位数算多少交给题，卡里只锚「写回需脏位」这个结论。

半导体存储器（SRAM/DRAM/Flash） 中频 对比表

SRAM vs DRAM 对比：

特性	SRAM	DRAM
存储元件	触发器（6 管）	电容（1 管 1 电容）
是否需刷新	**不需要**	**需要定时刷新**（电容漏电）
速度	快	慢
集成度	低	高
成本	高	低
典型用途	**Cache**	**主存**

- DRAM 刷新以**行**为单位，三种方式记名称：**集中刷新**（有死区）/ **分散刷新**（无死区但速度减半）/ **异步刷新**（折中，最常用）。
- 刷新不需要 CPU 干预，由存储器控制器自动完成。

ROM / Flash（非易失，断电不丢）：

类型	擦除方式与单位	关键点
EPROM	紫外线、**整片**擦除	慢，擦写次数有限
EEPROM	电擦除、**字节**级	慢，适合小规模参数
Flash	电擦除、**块**级	快，U 盘 / SSD / BIOS 的介质

- **Flash 三个高频结论**：① **读快写慢**（读写不对称）；② **写前必须先擦除**；③ 擦除以**块**为单位——和 EEPROM 的**字节**级擦除对着记。
- RAM 和 ROM **都支持随机访问**，别被「只读存储器」的名字骗成顺序存取。

多体交叉编址（只留结论，不做周期计算）：

编址方式	体号取地址的	连续地址落在	带宽
高位交叉（连续编址）	高位	同一个体内	不提升
低位交叉	低位	分散到各个体	提升

- **只有低位交叉能提高带宽**：相邻地址落到不同体上，多个体可**并行/流水**访问；高位交叉是**顺序扩容**，读完一个体才轮到下一个。
- 低位交叉按「模 m」编址，m 个体轮流启动，一个存储周期内可向 CPU 送出 m 个存储字。

一句话抓住根本差异：**SRAM 用触发器、DRAM 用电容**。电容会漏电，所以 DRAM 必须「定时刷新」（周期性重新充电恢复数据），SRAM 是双稳态触发器，只要不断电信息就一直保持、根本不需要刷新——这是两者所有差异的源头。

怎么记不混（顺着「电容」推）：电容小、做得密 → DRAM 集成度高、便宜 → 拿来做容量需求大的**主存**；触发器电路复杂、占面积 → SRAM 集成度低、贵、但快 → 拿来做容量小但要极快的 **Cache**。把「快/贵/小 = SRAM = Cache」「慢/便宜/大 =

DRAM = 主存」绑成两组，不会记反。

三种刷新方式只背名称和一句定性结论：

- **集中刷新**：在刷新周期末尾集中刷一遍，期间 CPU 不能访存 → 存在「**死区**」。
- **分散刷新**：每个存取周期后半段刷一行，**无死区**，但存取周期翻倍、速度减半。
- **异步刷新**：把刷新均匀分散到整个周期，死区短、最常用。

  刷新周期、刷新间隔（= 刷新周期 / 行数）、死区长度等数值计算交给题，卡里只留方式名称 + 定性对比。

ROM / Flash 这半边，顺着「擦除粒度」记：EPROM 要紫外线照整片 → EEPROM 进步到电擦除、能按**字节**改 → Flash 再快一步，但代价是退回**块**级擦除。所以 Flash 快在读、慢在写：写之前得先把整块擦干净，这就是「读写不对称」和「写前先擦除」的由来，也是它能当 U 盘 / SSD / BIOS 介质却当不了主存的原因。

> 「Flash 块级擦除 vs EEPROM 字节级擦除」是这一格最爱考的对比；SSD 的磨损均衡、闪存翻译层等细节归 [固态硬盘 SSD] 那张卡，这里不重复。

为什么只有低位交叉提带宽（顺着「访存是连续的」推）：cache 按块调入、DMA 按块传送，CPU 要的从来不是零散一个字，而是一串连续单元。低位当体号，连续地址就被摊到不同体上，各体错开时间轮流启动、流水地把数据送出来，一个存储周期内能送出好几个存储字；高位当体号，一整块连续地址全压在同一个体里，这个体读完才轮到下一个，并行无从谈起。所以高位交叉解决的是「容量不够」，低位交叉解决的才是「速度不够」——这两句是这一格最爱考的判据。

> 模数 m 取多少、存取周期与总线传输周期怎么配、几个体才不冲突、地址如何划分位段，全是数值代入，归题；卡里只留「低位交叉提带宽、高位交叉只扩容」这一条结论。

存储器层次结构 冷门 易错辨析

存储金字塔（从上到下）：**寄存器 → Cache → 主存 → 外存**。

越往上（越近 CPU）	越往下
速度越快	速度越慢
容量越小	容量越大
每位价格越高	每位价格越低

- Cache-主存层次：弥补 CPU 与主存速度差，**硬件**自动管理。
- 主存-辅存层次：弥补主存容量不足，**OS（软件）** 管理。

时间局部性 vs 空间局部性（正一反辨析）：

	含义	典型例子
时间局部性	刚访问过的，近期还会**再次访问**	循环里的循环变量 / 计数器
空间局部性	访问过地址**附近的**也会被访问	数组顺序遍历 / 顺序执行的指令

Cache 与虚拟存储器正是靠局部性才有效。

两级层次的类比与差异（Cache-主存 vs 主存-辅存）：

	Cache-主存	主存-辅存
解决什么	CPU 与主存的速度差	主存容量不足
谁来管	硬件自动完成	OS（软件）与硬件共同完成
传送单位	块（主存块）	页（页面）
不命中时	对程序员透明，硬件自动调块，程序无感	触发缺页异常，转 OS 调页后再重执行

- 两级的思路同构：都把下层的一部分内容缓存到上层，都要解决交换单位大小、映射、替换、写策略四个问题。
- 差异的根源是代价：缺页损失远大于 Cache 缺失，所以页比主存块大得多，且必须用全相联映射 + 回写写策略把缺失率和写外存次数压到最低。

存储金字塔记三句：**越往上越快、越小、越贵、越近 CPU**，方向永远一致，别记反。寄存器、Cache、主存 CPU 都能直接访问，外存不能——数据必须先调入主存。整套体系的目的，是让 CPU "看到"一个速度接近最快层、容量接近最大层、成本接近最低层的存储器。

怎么把两种局部性钉死不混：

- **时间**局部性盯的是「**同一个**地址」——刚用过的数据近期还会用，关键词是「再次、重复」，循环变量每轮都读就是它。
- **空间**局部性盯的是「**邻近**地址」——用了某地址，它周围的也快要用，关键词是「附近、相邻」，数组一个挨一个顺序访问就是它。

一句口诀区分：**时间盯「同一处反复」，空间盯「邻近一片」**。Cache 把刚访问的块（含其相邻数据）整块调入，就是同时吃这两种局部性——这是选择题最爱设的辨析陷阱，把哪个例子对应哪种局部性背反就丢分。

两级层次为什么能类比，又必须分清：金字塔上相邻的两级都是「上层当下层的缓存」——Cache 缓存主存的内容，主存则缓存外存的内容，所以块大小、映射、替换、写策略这四个问题两边都要回答。但它们被造出来的**目的相反**：Cache-主存这一级补的是**速度**（主存跟不上 CPU），主存-辅存这一级补的是**容量**（主存装不下整个程序）。目的一变，实现方式跟着全变。

最容易在选择题上丢分的是**「谁管 + 程序感不感知」这一对**。Cache 的判定、调块、替换必须在一条指令的执行过程中做完，慢一点指令就停摆，因此只能由硬件电路实现——这就是「**Cache 对程序员透明**」的由来：写程序时根本不用管数据此刻在 Cache 还是主存里。而缺页要去外存搬数据，慢到无法在指令内部消化，只能中止当前指令、转去执行操作系统的缺页异常处理程序，把页调进主存、改好页表项，再回来重新执行这条指令——**这一级要靠 OS 软件介入，不是纯硬件**。一句话记法：**Cache 不命中程序不知道，缺页则要惊动操作系统**。

差异的量级感也顺手记一下：外存比主存慢几个数量级，一次缺页的代价远大于一次 Cache 缺失，所以主存-辅存这级宁可把粒度放大（页远大于主存块）、把命中率抬到极致（全相联，任何虚页可放任意页框）、把写外存次数压到最少（回写而非全写）。

> 页表结构、TLB、页面置换算法（FIFO/LRU/CLOCK）那一套属于操作系统的虚拟内存部分，这里只锚「两级层次谁管什么、粒度多大、不命中谁来兜」，不重复展开。
>
> 各级平均访问时间、命中率、块大小/页大小的具体换算属于计算，交给题和可视化。卡里只锚金字塔方向 + 两种局部性的辨析。

固态硬盘 SSD（计组视角） 冷门 定义锚点

SSD（固态硬盘）以闪存（Flash）为介质，无机械结构。

- **读、写以「页」为单位，擦除以「块」为单位**（一块含多页）。
- **写前必须先擦除**（"先擦后写"）：要改某页的数据，需整块擦除后才能重写。
- 优点：无机械部件 → **随机读极快、抗震、低功耗、无噪声**。
- 缺点：**每块擦写次数有限**（寿命问题）→ 需 **磨损均衡（wear leveling）**，把擦写分摊到各块，避免个别块提前损坏。

SSD 用闪存芯片阵列存数据，没有磁盘那样的盘片、磁头、马达，所以不用寻道也不用等旋转，随机访问延迟极低、还抗震省电。但闪存有个硬约束：**读写的最小单位是「页」，而擦除的最小单位是更大的「块」**，且单元写之前必须先被擦成空白态——这就是「先擦后写、读写按页、擦除按块」三件套。

为什么随机写偏慢、为什么要磨损均衡：改一个已写过的页，不能就地覆盖，往往要把整块的有效数据搬到新块再写，擦除本身也比读写慢，所以随机写不如随机读快。又因为每个块能擦写的次数有限，如果总往同几个块上写，它们会先坏掉——**磨损均衡**就是让控制器把写入分散到擦写次数少的块上，拉平各块寿命。

> 本卡是**计组的介质特性视角**，与 OS 的 `ssd-management` 分列：FTL（闪存翻译层）的地址映射、垃圾回收、I/O 调度等属于 OS 语义，本卡不写，防止跨科泄漏。擦写寿命的具体次数、SSD 容量等数值交给题，卡里只锚介质特性结论。

指令系统 2 张

寻址方式 高频 对比表

各寻址方式——**含义 + 有效地址 EA 来源**（具体求值靠题）：

寻址方式	操作数 / 有效地址 EA	特点
隐含	操作数地址**隐含在操作码中**，指令不显式给地址	缩短指令字长；如单地址指令另一操作数隐含在 **ACC**
立即	操作数**就在指令里**（不是地址）	不访存取数，立即数位宽受指令字长限
直接	EA = A（地址码即有效地址）	地址范围受 A 位数限
间接	EA = (A)，A 指向**存有效地址的单元**	灵活、支持指针，多访存一次
寄存器	操作数在寄存器中	不访存，最快
寄存器间接	EA = (R)，寄存器存的是地址	取操作数只访存 1 次，比间接少访存一次
相对	EA = (PC) + A	A 为偏移，**转移指令**用，位置无关
基址	EA = (B) + A	基址寄存器 **B 由系统给定**，面向**程序重定位**
变址	EA = (X) + A	变址寄存器 **X 由用户改**，面向**数组 / 循环**
堆栈	操作数在**栈顶**，地址由栈指针 **SP** 隐含给出	零地址指令（如 `ADD` 弹栈顶两数运算再压回）；SP 自动增减

注：A = 指令中的地址字段，(R) = 寄存器 R 的内容。

- **基址 vs 变址（最爱混的钉死）**：基址 = 系统设定基址值不变、偏移 A 也固定 → 重定位；变址 = 偏移 A（数组首址）固定、变址值 X 由用户改 → 遍历数组。「谁在变、为谁服务」正好相反。
- 相对寻址基准是 **PC**，取指时 PC 已指向下一条，故目标 = 下一条指令地址 + 偏移。

寻址方式的考点分两层：**「这是哪种寻址 / EA 怎么来」是定性结论（背），「代入数算出具体 EA」是计算（归题）**。这张卡只锚每种方式的含义与有效地址来源。

怎么记不混：

- **隐含 / 堆栈是「不显式给地址」的两种**：隐含把地址藏进操作码（单地址指令的另一操作数默认在 ACC），堆栈把地址交给 SP（零地址指令从栈顶取数）——都靠"约定"省掉地址字段、缩短指令，别漏掉它俩。
- 一条主线按「操作数离 CPU 多远」排：立即（在指令里）→ 寄存器（在寄存器）→ 直接 / 寄存器间接（一次访存）→ 间接（两次访存）。越往后访存越多、越灵活。
- **基址和变址是镜像**：两者 EA 都是「寄存器 + 偏移 A」，区别只在谁变、干什么——
  - 基址：**B 由操作系统 / 系统设定**，程序里偏移 A 是相对起始的位置，用于多道程序的**重定位**；
  - 变址：**X 由用户（程序员）修改**，A 是数组首地址固定不变，X 当下标，用于**循环遍历数组**。
    记一句「基址挪程序、变址挪下标」。
- 相对寻址的坑：基准是 PC 不是某个数据地址，且 PC 已自增指向下一条，算目标地址要带上这一步。

「主存地址 2000H、偏移 06H，求转移目标地址」「EA = (B)+A 代值算」这类是有效地址求值，属计算，看下方可视化逐步推。卡里只留各方式的含义与 EA 来源。

CISC 与 RISC 低频 对比表

CISC vs RISC 多维对比：

维度	CISC（复杂指令集）	RISC（精简指令集）
指令数量	多（数百条）	少（几十条核心）
指令长度	变长	**定长**
寻址方式	多且复杂	少而简单
访存	指令可**直接访存**	**Load/Store**：只有 load/store 访存
控制器	微程序控制为主	**硬布线控制**为主
通用寄存器	少	多
指令周期	不定（差异大）	规整、利于流水线
典型代表	x86	ARM / MIPS / RISC-V

核心一句：**RISC 用简单、定长、规整的指令换流水线效率**（不是单纯「指令少」）。

- RISC 关键三特征：**简单 + 定长 + Load/Store**，只有 load/store 能碰内存，其余运算都在寄存器间完成。
- CISC 兼容性强、靠复杂指令减少程序条数；RISC 靠编译器优化 + 硬件流水线提吞吐。

CISC / RISC 对比是单点辨析型选择题（「关于 RISC 错误的是」），重点抓两套特征互为对照即可。

怎么记不混：

- 从一个原点推开：RISC 的灵魂是「**指令简单定长**」，由它派生出其余特征——指令定长 → 译码规整 → 易流水线；指令简单 → 控制逻辑可控 → 用硬布线（快）；运算只在寄存器间做 → 必须先 Load 进寄存器、算完 Store 回去 → 所以通用寄存器要多。
- CISC 几乎处处相反：变长指令、多寻址方式、可直接访存、微程序控制、寄存器少。记「CISC 复杂换条数少、RISC 简单换流水快」。
- 易错点：RISC 不等于「指令越少越好」，关键是简单、定长、Load/Store 结构；现代 x86 对外是 CISC 接口、内部把指令拆成 RISC 风格的微操作执行（混合架构），但 408 仍按经典两分法考。

  这张卡是纯结论对照、无计算。具体指令的微操作 / 流水线加速比属其他 KP，不在此卡。

中央处理器 4 张

异常与中断（计组视角） 高频 易错辨析

异常 vs 中断（这是辨析点）：

	异常（内中断）	**中断（外中断）**
来源	CPU **内部**，由**当前指令执行**触发	**外部** I/O 等外设触发
时序	**同步**（与指令相关）	**异步**（与当前指令无关）
举例	除零 / 缺页 / 非法指令	I/O 完成、时钟、外设请求
检测时机	指令执行**过程中**	指令执行**结束后**
可屏蔽	偏不可屏蔽	多为可屏蔽（NMI 除外）

中断响应过程（中断隐指令，硬件自动完成）——次序不能乱：

> **① 关中断 → ② 保存断点（PC / PSW）→ ③ 送中断向量（引出中断服务程序入口）**

随后才是**中断服务程序（ISR，软件）**：保护现场 →（开中断）→ 处理 → 恢复现场 → 中断返回。

- **陷阱 1**：中断隐指令由**硬件自动完成，不是一条真正的机器指令**（程序里看不到它）。
- **陷阱 2**：**保存断点（PC/PSW，硬件做）≠ 保存现场（通用寄存器，软件 ISR 做）**，两者不同主体、不同时机。
- **关中断必须排最前**：它给「保存断点」这段上锁，否则保存到一半来新中断会覆盖刚存的断点。

这张卡两个核心：异常和中断怎么分、中断响应那三步的次序与软硬件分工。

异常 vs 中断怎么记：盯「触发源 + 时序」一对——异常是**当前这条指令自己**捅出来的（除零、缺页、非法指令），所以**同步、执行过程中**就发现；中断是**外面的设备**找上门，跟你正执行哪条指令无关，所以**异步、要等当前指令执行完**才检测响应。一句话「异常是内鬼同步、中断是外客异步」。

中断响应三步为什么是这个序：

- **先关中断**——给后面「保存断点」上临界区锁。若不先关，保存 PC 到一半又来一个新中断，新中断的入栈会把刚保的断点覆盖掉，回不去原程序了。
- **再保存断点（PC/PSW）**——记下「回到哪、当时状态如何」，这是硬件存的。
- **后送中断向量**——由中断类型号引出中断服务程序入口地址，跳过去。
- 这三步合称**中断隐指令**，全程由**硬件**完成，**不是一条能写在程序里的真指令**。它之后才轮到软件的 ISR 去「保护现场（通用寄存器）」。**保存断点（硬件）和保护现场（软件）是两件事，别混。**

⚠️ 减法：中断优先级判定、中断屏蔽字（响应优先级 vs 处理优先级）的具体计算属计算 / 推导，交给题。卡里只锚异常中断辨析、三步次序与软硬件分工。

> 「除零 / 缺页 / 系统调用处理后是否返回当前指令重新执行」「中断响应哪一步必须由硬件完成」这类辨析正是上方两道真题考的点；屏蔽字优先级换算看下方可视化。

指令流水线 高频 对比表

① 三类冒险（相关）· 成因 → 对策

冒险	别称	成因	主要对策
结构冒险	资源冲突	同一周期多条指令争用同一部件（如取指与访存都要用存储器）	指令 cache 与数据 cache 分开；寄存器读口写口分开
数据冒险	数据相关	后面指令要用前面指令的结果，而结果还没产生	转发（旁路）；解决不了时插气泡/nop 阻塞
控制冒险	分支冒险	转移指令（含调用返回、异常中断）改变了指令执行顺序	分支预测（静态/动态）；延迟分支填延迟槽；阻塞

- 基本流水线（不乱序）中的数据冒险**全部是 写后读（RAW）**：前面写、后面读同一寄存器。
- 转发不是万能的： `load` 指令后紧跟用它结果的指令，即 load-use 冒险，**必须停顿**（编译器调序或插 nop / 硬件阻塞）。
- 延迟分支属**静态调度**：把分支前无关的指令调到 分支延迟槽 里执行，填不满用 nop 补。

② 超流水线 vs 超标量 vs 超长指令字（VLIW）

技术	做法	一个周期发射	理想 CPI	谁做打包/调度
超流水线	把流水段划得更细、增加流水线级数	1 条	CPI 仍为 1	—
超标量	多套部件并行，多发射	多条	CPI<1（用 IPC 衡量）	动态多发射，硬件定
VLIW 超长指令字	多个操作打包进一条长指令	多条（打包时定死）	CPI<1	静态多发射，编译器定

> 超标量与 VLIW 都是**多发射**，分界只有一条：**打包由硬件动态做（超标量）还是由编译器静态做（VLIW）**。超流水线不属于多发射，它只是把流水线做得更深。

流水线把一条指令拆成若干段重叠执行，前提是"每拍都能往前推一步"。**冒险就是推不动的三种理由**：部件不够用、数据还没算出来、下一条该取哪一条还不知道。三类冒险的名字直接对应这三种理由，成因记住了对策就不用背。

小节① 三类冒险怎么对号入座：

- **结构冒险**是硬件层面的抢资源。经典场景是同一个时钟周期里，一条指令在取指、另一条在访存，都要用存储器。对策也就落在"把资源拆开"上——指令存储器与数据存储器分离（现代机器天然满足，L1 一般就是指令 cache 与数据 cache 分离），寄存器堆的读口和写口分开用。
- **数据冒险**是数据还没准备好。转发（旁路）的思路是：结果其实在流水段寄存器里已经算出来了，不必等它写回寄存器堆，直接从流水段寄存器拉到 ALU 输入端。所以绝大多数写后读相关都能被转发消化掉——**但"所有数据冒险都能靠转发解决"是错的**， `load` 的数据要到访存段结束才拿到，紧跟着用它的指令在执行段就要读，时间上倒挂，只能停顿。
- **控制冒险**是不知道下一条取哪条。消极办法是插气泡/插 nop 等分支结果出来；积极办法是分支预测，猜错了再把已经进流水线的错误指令冲刷掉。延迟分支是另一条路：编译器把分支前与分支无关的指令搬到分支后面的延迟槽里，反正它执行与否都不影响正确性。

- 一句话辨析：**转移指令引起的是控制冒险，不是数据冒险**；而"数据旁路（转发）"本身是解决手段，不是冒险来源——这一对是选择题最爱设的反向陷阱。

小节② 三种高级流水线技术怎么记不混：

- 先分两类：**超流水线只把流水线"做深"，一个周期仍然只发射一条指令**，CPI 不变，靠缩短时钟周期提吞吐；超标量和 VLIW 是把流水线"做宽"，一个周期发射多条，CPI 才可能小于 1（这时用 IPC 更顺手）。
- 再分超标量和 VLIW：两者都是多发射，唯一分界是**打包的时机与主体**——超标量由处理器硬件在运行时决定这一拍发几条、发哪几条；VLIW 由编译器在编译时就把能并行的操作固定打包成一条长指令，硬件照单执行、凑不满就填 nop。所以 VLIW 对编译器的依赖极重。
- 顺带一个易混：多发射并不能消除冒险，反而会放大——一拍里跑多条指令，一旦阻塞被拖住的指令也成倍增加。

  时空图、加速比、效率、吞吐率、流水线时钟周期取最慢段这类**计算全部归题**（配合可视化练更快）。这张卡只锚三类冒险的成因与对策、以及三种高级流水线技术的定义分界。

控制器 中频 对比表

硬布线 vs 微程序控制器：

对比项	硬布线控制器	微程序控制器
实现方式	**组合逻辑电路**（无存储器）	微指令存于**控制存储器 CM（ROM）**
速度	**快**（直接出控制信号，无查表）	较慢（需从 CM 读微指令）
规整 / 扩展	不规整、逻辑复杂	**规整、易扩展**
修改灵活性	**难**（要重新设计电路）	**易**（改 CM 内容即可）
适用	RISC（指令简单）	CISC（指令复杂）

控制信号 = f(操作码, 时序节拍, 状态条件)（硬布线即此布尔函数的电路实现）。

- 微指令操作控制字段的三种**编码方式**（记名称即可）：**直接编码 / 字段直接编码 / 字段间接编码**。直接编码快但字长大；字段直接编码用译码器压缩字长；字段间接编码再压缩、译码更复杂。
- 一句话：**硬布线快但改难（RISC）、微程序慢但灵活（CISC）**。

控制器对比是稳定的选择题点（「硬布线相对微程序的特点」）。重点是「速度 vs 灵活」这对取舍，以及各自配 RISC / CISC。

怎么记不混：

- 抓一对核心矛盾：硬布线是**电路硬连**——信号一步出来所以**快**，但要改就得重新布线所以**改难**；微程序是**查表（读 CM）**——多一道读 ROM 所以**慢**，但改指令只要改 ROM 内容所以**灵活**。速度与灵活恰好对立。
- 配对记忆：指令简单 → 电路可控 → 硬布线 → RISC；指令复杂 → 硬连太庞大 → 微程序 → CISC。
- 控制存储器 CM 是 CPU 内部的 ROM，存微程序，和主存互相独立、不占主存地址空间；取指令从主存取、取微指令从 CM 取。
- 微指令编码只背三种名称与「越压缩字长、译码越慢」的趋势。

  「下地址字段位数」「字段直接编码操作控制字段最少多少位」「微指令最小字长」这类是位数 / 字长计算，属计算题，看下方可视化。卡里只留两种控制器的特点对照与编码方式名称。

多处理器基本概念 冷门 对比表

Flynn 分类（按指令流 × 数据流数量）：

类型	指令流	数据流	典型代表
SISD	1	1	传统单处理器
SIMD	1	多	向量处理器、GPU
MIMD	多	多	多核 / 多处理器系统
MISD	多	1	理论存在，**实际几乎没有**

几个辨析名词：

- **SMP（对称多处理）**：多处理器共享**单一物理地址空间**，靠访存指令读写共享变量通信。
- **多核处理器**：多个核集成在同一芯片，每核独立跑一个线程 → **物理上真正并行**。
- **硬件多线程（含超线程 SMT）**：每个线程配独立寄存器组 + PC，切换不用读写内存；超线程 = 一个物理核维护两套线程状态、共享执行单元，是「逻辑多核」**不等于物理多核**。

这张卡是 low tier 的概念辨析卡，全是名词对照、无计算。重点抓 Flynn 四类怎么分，以及「多核 / SMP / 硬件多线程」几个易混名词的定义边界。

怎么记不混：

- Flynn 就数「指令流几条 × 数据流几路」：SISD 是 1×1（传统单核）；SIMD 是 1 条指令管多路数据（一条指令同时算一批数，向量 / GPU）；MIMD 是多指令多数据（多核 / 多处理器，主流并行）；MISD 多指令处理同一数据，**几乎不存在**，常作干扰项。
- **多核 vs 硬件多线程**：多核是**真有多个物理核**，同时各跑一线程；硬件多线程（含超线程）是**单核里塞多套线程状态部件**，靠共享执行资源交错 / 同时发射，是「看起来像多核」的逻辑并行，不是物理多核。考点常考「硬件多线程是否只能用于多核」——不是，单核也能做。
- SMP 关键词是「共享单一物理地址空间」，所有处理器都能用普通访存指令访问全部内存。

⚠️ 超纲红线：多处理器的**缓存一致性协议（MESI 等）不展开**，408 计组本 KP 只到「存在一致性问题、由硬件保障」的概念层；UMA/NUMA 延迟差异、一致性协议状态机不背。

> 这张卡纯定性辨析，无计算中间值。本 KP 命题不高频，作为概念兜底卡。

总线 3 张

总线基本概念与组成 高频 定义锚点

总线 = 连接各部件的**共享传输线，分时复用**（同一时刻只能一个主设备发送）。

三个分类维度（钉死）：

维度	类别
按功能	数据总线（双向）/ 地址总线（单向，CPU 发出）/ 控制总线
按层次	片内总线（CPU 内部）/ 系统总线（CPU-主存-I/O）/ I-O 总线（通信/外总线，如 USB、SATA）
按传输	串行 / 并行

- **带宽公式（只记形式，代入计算归题）**：带宽 = 总线宽度（位）÷ 8 × 每秒传输次数（B/s）。
- **猝发（突发）传送**：给一个地址后**连续传多个数据**，提高带宽（TOP KP 高频点）。
- 数据总线宽度决定单次传输量；地址总线位数决定寻址空间大小。

总线本质是一组**公共导线**，多个部件分时共享：任一总线周期内只能有一个主设备控制总线、选中一个从设备一对一通信——这正是需要总线仲裁的根本原因。

怎么记不混：

- **按功能三总线方向**：地址总线**单向**（CPU 发出地址去寻址）；数据总线**双向**（传的"数据"含操作数、指令、中断类型号，凡走数据通路的都算）；控制总线有出有入。
- **按层次三总线由内到外**：片内（芯片内部，最快）→ 系统（机箱内核心，408 出题最多）→ I-O / 通信（连外设或其他机器，USB / SATA / 以太网）。
- **提高总线数据传输率的三条路**（命题点）：① 增加总线宽度 ② 提高工作频率 ③ 支持猝发传送。注意"地址/数据线复用"是**省线**而非提速。
- **总线工作频率可以是时钟频率的整数倍**：DDR 在时钟上升沿、下降沿各传一次，工作频率 = 2 × 时钟频率，做题先看清"每个时钟周期传几次数据"。

  总线带宽具体代入（宽度÷8 × 频率 × 每周期次数）、突发传送有效带宽（含地址/等待周期）、寻址空间 2^n 这些演算都归题与可视化。卡里只锚公式形式 + 分类 + 猝发定性。

总线仲裁 冷门 对比表

集中仲裁三方式对比（一个中央仲裁器决定谁占用总线）：

方式	优先级	仲裁速度	控制线数	故障敏感
链式查询	**固定**（离控制器近者高）	慢	**最少**（3 条，与设备数无关）	高（断链全瘫）
计数器定时查询	可变（计数初值可调）	中	中（$\lceil \log_2 n \rceil + 2$）	中
独立请求	灵活可编程	**最快**	**最多**（2n+1）	低

一句记法：**链式省线慢、独立耗线快**；计数器居中。

总线同一时刻只能被一个主设备控制，多个主设备竞争时由仲裁选出一个授予控制权（BR 总线请求 → BG 总线授权 → 占用 → 释放）。

怎么记不混：

- **链式查询（菊花链）**：所有设备共享一条 BR，BG 信号沿链**串行**依次传递，谁离控制器近谁优先级高、想用就截获 BG。线最少（BR + BG + BS 忙信号 = 3 条），但优先级写死（低优先级可能饿死）、BG 串行传递所以慢、链路中某设备坏了后面全收不到 BG。
- **计数器定时查询**：控制器把计数值广播给所有设备，编号匹配且有请求者获权。计数从 0 开始 = 固定优先级，从上次停处继续 = 公平轮转——所以优先级"可变"。
- **独立请求**：每个设备各拉一对 BR/BG 到控制器，控制器**同时看到所有请求**直接选，所以最快、优先级最灵活、不怕单点故障，代价是线最多。

  各方式控制线数的具体计算（如 16 设备独立请求 = 2×16+1 = 33 条）归题。卡里只锚"谁最少线、谁最快、谁优先级固定"的定性对比。

总线定时 冷门 对比表

总线定时（主从如何协调传输节奏）：

方式	时钟	协调手段	特点
同步	统一时钟	固定时钟边沿采样	简单、速度快、可精确算带宽；但**须按最慢设备定时钟**、不适应速度差异大的设备
异步	无统一时钟	请求/应答**握手**	灵活适配不同速度设备；控制复杂、较慢
半同步	同步 + WAIT	同步为主，从设备跟不上拉 WAIT 插等待周期	折中，兼顾简单性与灵活性

异步三种握手（互锁程度递增 → 越可靠越慢）：

	等待	速度	可靠性
不互锁	0 次	快	低
半互锁	3 次	中	中
全互锁	4 次	慢	高

定时方式解决的核心问题：主从设备如何**协调传输节奏**，确保数据在正确时刻被采样。

怎么记不混：

- **同步 vs 异步的根本差异**：有没有统一时钟。同步靠"约定好第 k 个时钟周期取数"，简单快但全总线被最慢设备拖住；异步靠"我发请求、你回应答"逐步握手，慢但能适配快慢不一的设备。
- **三种握手的"互锁"= 撤销时是否互相等待**：全互锁每步都等对方确认（请求/应答/撤请求/撤应答四步全等），最稳最慢；半互锁主设备收到应答就撤请求、不等从设备撤应答；不互锁双方各自定时延后自动撤销、谁都不等，最快最不可靠。
- **半同步 = 同步 + WAIT 信号**：默认走同步，从设备慢就拉低 WAIT 让主设备插入等待周期，就绪后释放——是同步的"打补丁"版。

  同步总线带宽 = 宽度÷8 ×(1/(k×T))、突发传送有效带宽、读一个主存块要几个时钟周期这类演算全归题。卡里只锚四种方式的定性对比与握手次数。

输入输出系统 2 张

DMA 方式 高频 对比表

① 四种 I/O 方式 · CPU 干预程度对比（干预由多到少）

方式	数据是否经 CPU	传输单位	CPU 介入	CPU 干预
程序查询	是	字/字节	**全程忙等轮询、串行**	最多（最低效）
中断	是	字/字节	每传一次中断一次、可与外设并行	中
DMA	**否**	数据块	**仅传送前后干预**	少
通道	否	多个数据块	一次启动管多块	最少

- 数据走向：程序查询/中断都**经 CPU 经程序传送**；DMA 数据**由 DMA 控制器直接在主存↔外设间传，CPU 不搬运**。
- 响应时机：中断在**指令执行结束后**响应；DMA 在**存储周期结束后**响应（更细粒度）。
- DMA 传完仍**靠中断**通知 CPU（中断没被取代，只是从"每字一中断"降为"整块一中断"）。

② DMA 三种访存方式 + 周期窃取陷阱

停止 CPU 访存 / **周期挪用（周期窃取）** / 交替访存。

> **钉死易错点**：DMA **周期窃取时 CPU 并未停机**，只是被"窃取"了一个存储周期（CPU 与 DMA 同时请求时 **DMA 优先**）。

DMA 的核心思想：数据传输**不经过 CPU**，由 DMA 控制器（DMAC）直接控制总线在设备与主存间搬运，CPU 只在开始前设参数、结束后响应中断。

小节① 四种方式怎么记不混：

- 主线是一条递减链：**程序查询 → 中断 → DMA → 通道，CPU 参与度逐级降低**。程序查询 CPU 死等（忙等轮询，效率最低）；中断让 CPU 等待期间能干别的，但每传一个字仍要 CPU 经程序搬一次；DMA 连搬运都交给 DMAC，CPU 只管"传前设参数、传后收中断"；通道是"专做 I/O 的弱 CPU"，一次启动就把多个数据块管完，CPU 几乎不插手。
- 易混："中断方式的优势不是减少 I/O 的 CPU 开销，而是让 CPU 在等待期间去做有用的事"——这句是命题陷阱常考点。

小节② 周期窃取为什么 CPU 不停机：

- 三种访存方式按"对 CPU 影响"排：停止 CPU 访存（CPU 被完全阻塞）> 周期窃取（CPU 只被延迟一个存储周期）> 交替访存（CPU 无延迟，但要求存储器够快）。
- **周期窃取（周期挪用）是最常用方式**：DMA 每需传一个字就"偷"一个存储周期占总线、用完归还，CPU 程序仍在跑、只是该次访存被推迟一拍。所以"DMA 工作时 CPU 暂停"是错的——这正是章节点评点名的经典陷阱。
- CPU 与 DMA 同时请求访存时 **DMA 优先**：磁盘等外设有实时性（数据转过去就丢），CPU 让一拍代价很小。

DMA 传送字数（WC 位宽）、最大寻址空间（AR 位宽）、传输率/CPU 占比、总线请求次数这些**计算全归题**。卡里只留四方式干预程度对比 + 周期窃取定性辨析。

I/O 接口 中频 对比表

I/O 接口（设备控制器）= CPU 与外设之间的桥梁。

接口四大功能：地址译码（设备选择）/ 数据缓冲（速度匹配）/ 状态与控制（命令传递+状态反馈）/ 中断管理。

接口内寄存器统称 **I/O 端口**：数据端口（可读写）/ 状态端口（CPU 只读）/ 命令端口（CPU 只写）。

两种端口编址方式对比：

	独立编址	**统一编址（内存映射 I/O）**
地址空间	与内存**分开**	占用内存地址空间
用什么指令	**专用 I/O 指令 IN/OUT**	普通访存指令（MOV/LOAD/STORE）
占内存空间	否	是
区分内存/IO	控制信号（M/IO）	地址范围

外设速度、信号格式、控制逻辑都和系统总线不匹配，所以不能直挂总线，必须经 I/O 接口做速度匹配、信号转换、协议适配。

怎么记不混：

- **三种端口的读写方向**：状态端口报告"忙/就绪/错误"给 CPU 看 → CPU **只读**；命令端口装 CPU 下达的启停命令 → CPU **只写**；数据端口进出数据 → 可读可写。
- **两种编址记"占不占内存 + 用什么指令"**：独立编址 = I/O 有自己的地址空间、要用专门的 IN/OUT 指令、不占内存，但只有这两条指令、寻址方式少；统一编址（内存映射）= 把端口当内存单元、所有访存指令都能操作 I/O、寻址灵活，代价是吃掉一块内存地址、程序里不易分辨哪些地址是 I/O。
- 易错点：独立编址下 `MOV AX,[2000H]` 访问的是**内存**不是端口——MOV 是访存指令，独立编址里访端口只能用 IN/OUT。

  编址方式下"可用内存空间 = 总地址空间 − I/O 占用"这类地址计算归题。卡里只锚接口功能、三端口、两种编址的定性对比。

扫码回站内背这套卡
站内按记忆曲线排期，到期自动进「今日待背」